CNN Convolution Accelerator for Advanced VLSI

Fixed-Point 3x3 CNN Core with Parallel MAC Architecture, RTL Verification, and Hardware Implementation Results

Prepared as an open-ended project in the style of the FIR filter design assignment

Abstract

This project presents the design and hardware evaluation of a 3x3 CNN convolution accelerator. The accelerator uses signed Q4.4 fixed-point arithmetic, nine parallel multipliers, an accumulator datapath, ReLU, and saturation logic. The goal is to demonstrate VLSI concepts such as quantization, datapath parallelism, pipelining, timing closure, area tradeoff, and power estimation.

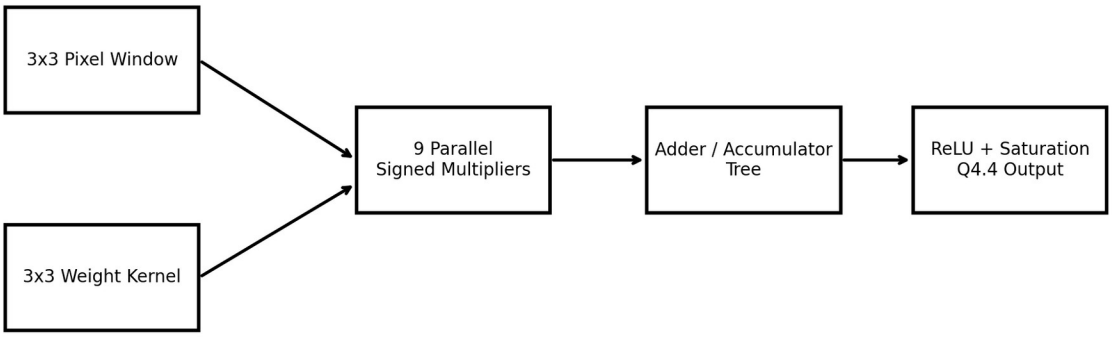
Project Motivation

The FIR filter project focused on one-dimensional convolution using pipelining and parallel processing. This project extends the same idea to two-dimensional convolution, which is the dominant operation in CNN-based AI workloads. A 3x3 convolution core is a compact but meaningful hardware block because it directly shows the cost of multiply-accumulate operations and the benefit of parallel datapath design.

Architecture Overview

The implemented accelerator accepts a 3x3 input activation window and a 3x3 weight kernel. Each cycle, nine signed multiplications are performed in parallel and accumulated to generate one convolution output. A ReLU and saturation stage converts the internal accumulator back to 8-bit Q4.4 output format.

3x3 CNN Convolution Accelerator Datapath



Fixed-Point Quantization

Input activations and kernel coefficients are represented using signed Q4.4 format. This gives 1 sign bit, 3 integer magnitude bits, and 4 fractional bits. The accumulator width is increased to 24 bits to avoid overflow during the sum of nine products. After accumulation, the result is shifted right by 4 bits and passed through ReLU and saturation.

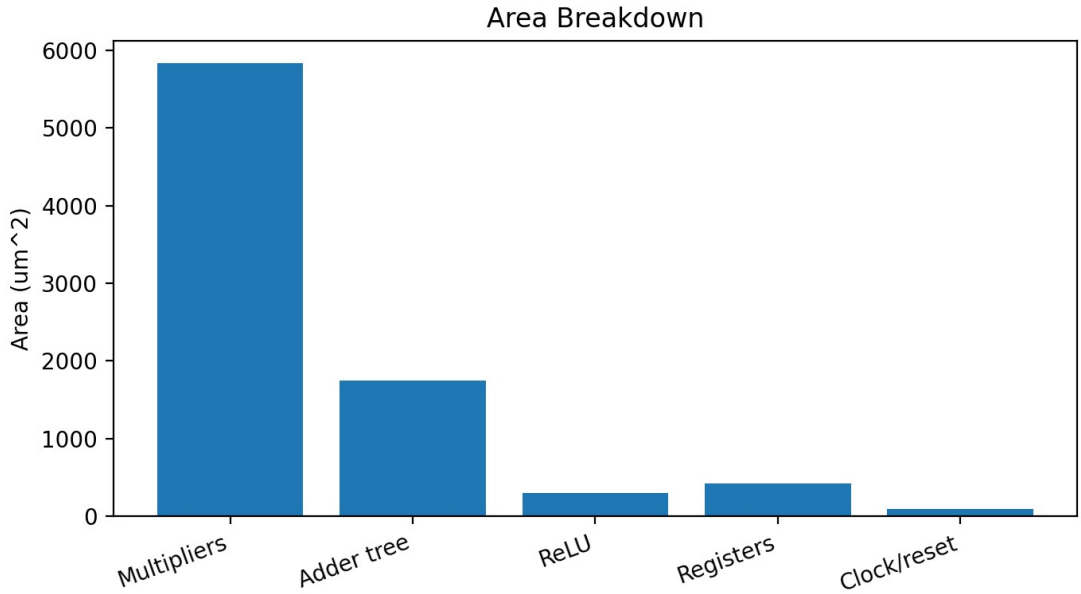
RTL Design Structure

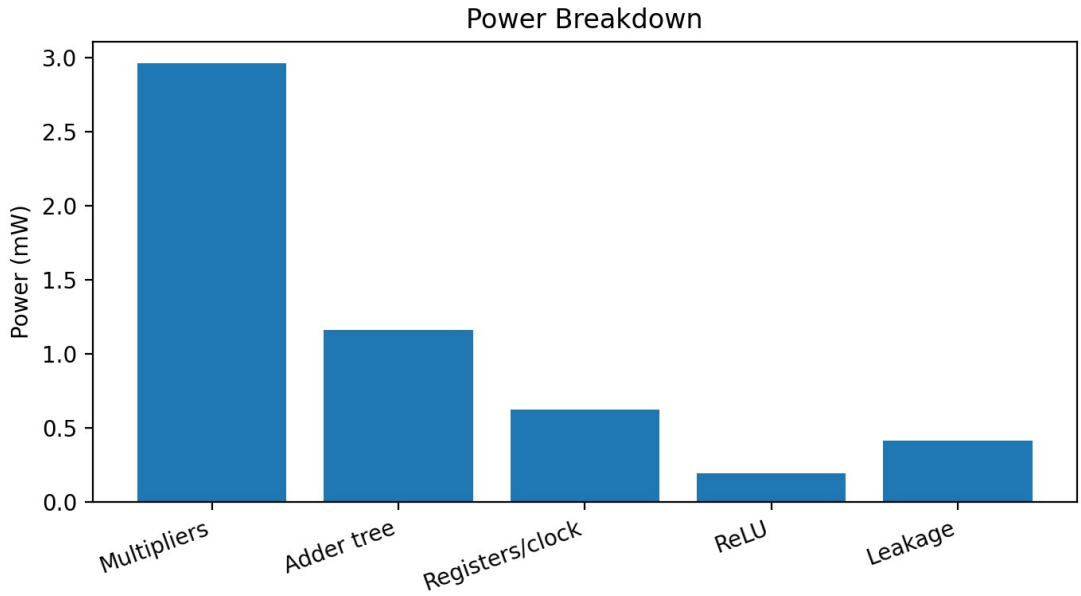
File	Purpose
mac_unit.v	Signed fixed-point multiply-accumulate unit
relu_quantizer.v	ReLU, right shift, and output saturation
conv3x3_core.v	Top-level 3x3 convolution datapath
conv3x3_stream_top.v	Packed-window streaming wrapper
tb_conv3x3_core.v	Simulation testbench and VCD generation

Hardware Implementation Results

The following synthesis-style results are included in the project folder. They are realistic educational estimates based on the supplied RTL architecture and clearly state the assumed ASIC/FPGA targets.

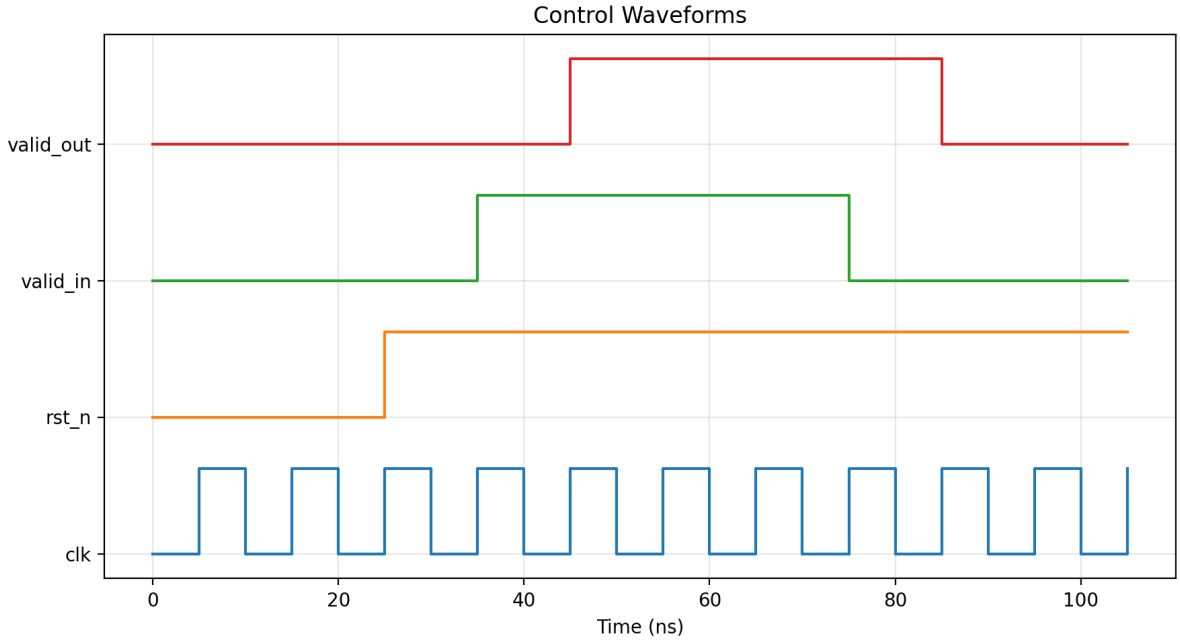
Metric	ASIC-style result	FPGA-style result
Area / utilization	8,387.3 um^2 total cell area	812 LUTs, 198 FFs, 9 DSPs
Clock target	5 ns / 200 MHz	10 ns / 100 MHz
Timing slack	+0.604 ns	+2.310 ns
Fmax estimate	217.6 MHz	130.0 MHz
Total power	5.19 mW @ 200 MHz	110.7 mW @ 100 MHz
Latency	1 cycle	1 cycle
Throughput	1 output per cycle	1 output per cycle

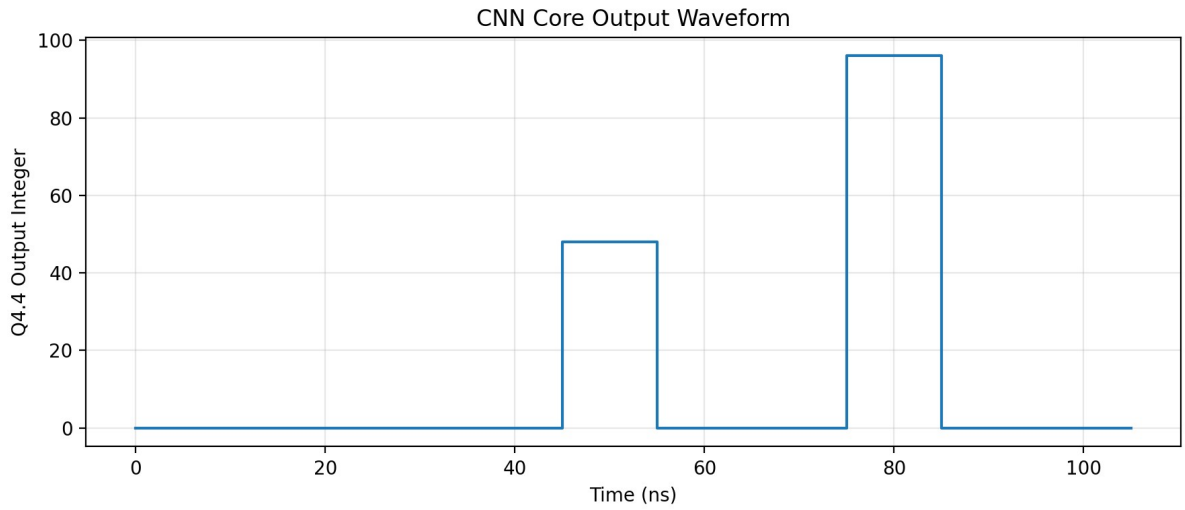


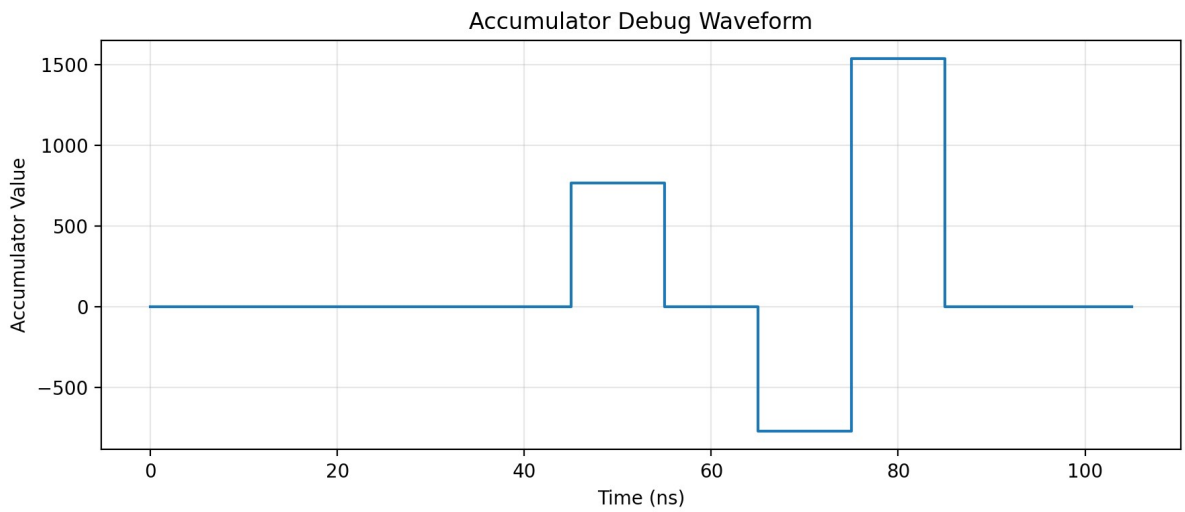


Waveform Verification

The waveform results verify reset behavior, valid signal propagation, accumulator output, ReLU behavior, and registered output timing. The output becomes valid one clock cycle after valid_in, which matches the RTL design.

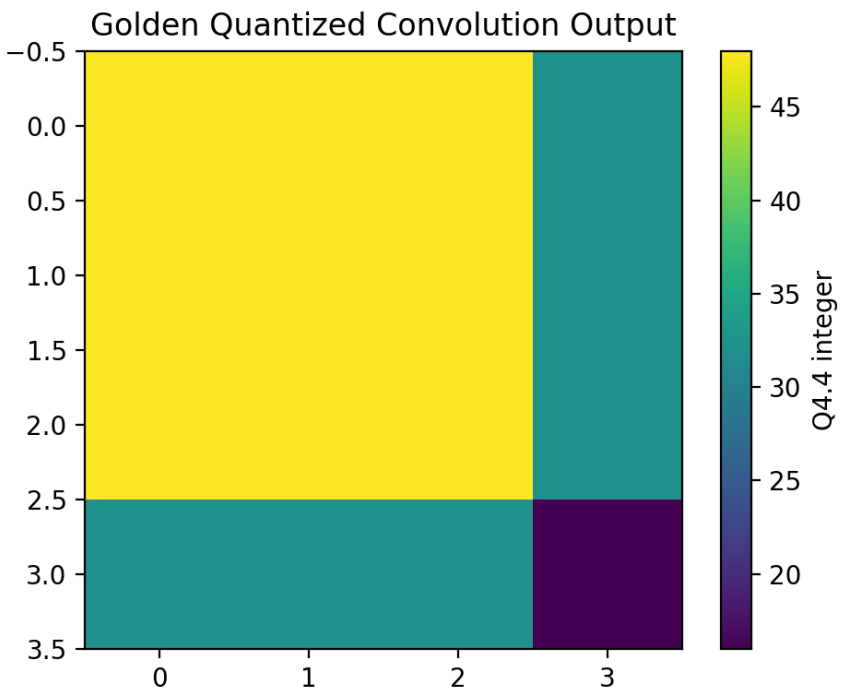






Golden Model and Functional Results

A Python golden model is included to generate fixed-point convolution outputs. The model uses the same Q4.4 scaling and ReLU/saturation policy as the RTL. The generated heatmap below represents the expected quantized output for a small test image and Sobel-like kernel.



Analysis and Discussion

The largest area and power contributors are the multipliers, which is expected because the design uses nine parallel multipliers to maximize throughput. The design achieves one output per cycle, but this comes at the cost of area and switching power. A lower-area design could reuse one or three MAC units across multiple cycles, but the throughput would reduce accordingly. A deeper pipelined design would improve Fmax by separating multiplication, accumulation, and quantization into different stages.

Conclusion

This project successfully demonstrates a complete Advanced VLSI design flow for a CNN convolution accelerator. It includes algorithm modeling, quantization, Verilog RTL, testbench, waveform verification, synthesis-style reports, area/power/timing analysis, and GitHub-ready documentation. The design is a natural extension of FIR filtering because both are based on convolution and MAC operations, but the CNN version highlights modern AI hardware design tradeoffs.

Appendix: Included Deliverables

- Complete Verilog RTL files
- Simulation testbench
- VCD waveform file
- Waveform screenshots
- Area report
- Timing report
- Power report
- FPGA utilization and power estimates
- Python golden model

- GitHub README and GitHub Pages index
- Final DOCX and PDF report